

COMPLETE GUIDE

# Exploratory Data Analysis

Master EDA from Zero to Hero

From the 21-Day AI Mastery Roadmap | Day 1: Machine Learning

6 Chapters • Python Code Examples • Titanic Dataset Walkthrough • Graph Reference

# Table of Contents

---

## **01 Introduction to EDA**

What is EDA and why should you care?

---

## **02 Importance of EDA**

What happens when you skip it

---

## **03 Data Types**

Numerical vs Categorical — the two families

---

## **04 Python Packages for EDA**

Your essential toolkit

---

## **05 List of Graphs**

Which chart for which question

---

## **06 Practical EDA**

Step-by-step Titanic walkthrough

---

## **Bonus: Google Colab**

Your free AI coding playground

---

SECTION 01

# Introduction to EDA

Imagine you just received a **giant box of LEGO pieces** — thousands of them, all mixed up. Before you start building anything, what do you do? You **look through the box first!** You check what colors are there, how many big vs small pieces, if any are broken, and what you could build with them.

That's exactly what EDA is.

EDA stands for **Exploratory Data Analysis** — the process of examining and visualizing a dataset to understand its patterns, spot anomalies, and extract insights *before* building any machine learning model.

It was pioneered by mathematician **John Tukey** in 1977, who said: "The greatest value of a picture is when it forces us to notice what we never expected to see."

EDA answers 4 basic questions:

| Question                     | What It Means             | Example                         |
|------------------------------|---------------------------|---------------------------------|
| What does my data look like? | Shape, size, column names | 891 rows, 12 columns            |
| Is anything missing?         | Empty cells, null values  | Age has 177 missing values      |
| Are there any patterns?      | Trends, correlations      | Women survived more than men    |
| Is anything weird?           | Outliers, errors          | Someone paid \$512 for a ticket |

**Think of EDA as being a detective with data.** You're investigating the crime scene (dataset) before making any arrests (building models).

SECTION 02

# Importance of EDA

Why can't we skip EDA and go straight to building AI models? Here's a real-life analogy:

Imagine a doctor who **doesn't examine a patient** before surgery. No X-ray, no blood test, no questions asked — just starts cutting. Scary, right? **Skipping EDA is the same thing.** You'd be building a model on data you don't understand.

| Without EDA               | With EDA                |
|---------------------------|-------------------------|
| Garbage in, garbage out   | Clean, trustworthy data |
| Hidden bugs in your model | Right features selected |
| Wrong conclusions         | Better model accuracy   |
| Wasted weeks of work      | Confident decisions     |
| MODEL FAILS               | MODEL SUCCEEDS          |

**Real example:** A company built a sales prediction model that was 95% accurate. But they never did EDA. Turns out, 40% of the data had the same date copied — their model was just memorizing duplicates. EDA would have caught that in 5 minutes.

SECTION 03

# Data Types

Every column in your data speaks a different language. Understanding data types is crucial because **each type needs different graphs and different analysis methods**. There are 2 main families:

## Family 1: Numerical Data (Numbers you can do math on)

| Sub-type   | Definition                    | Examples                                                | Key Property                |
|------------|-------------------------------|---------------------------------------------------------|-----------------------------|
| Discrete   | Whole numbers only, countable | Kids: 0, 1, 2, 3<br>Goals scored: 0, 1, 5               | Can't have 2.5 kids!        |
| Continuous | Any decimal value, measurable | Height: 5.7 feet<br>Temp: 98.6°F<br>Salary: \$52,340.50 | Infinite precision possible |

## Family 2: Categorical Data (Labels or groups)

| Sub-type | Definition              | Examples                                              | Key Property                 |
|----------|-------------------------|-------------------------------------------------------|------------------------------|
| Nominal  | Names with NO ranking   | Color: Red, Blue, Green<br>City: Dhaka, London, Paris | Red isn't 'higher' than Blue |
| Ordinal  | Categories WITH ranking | Rating: Bad, OK, Good<br>Size: S, M, L, XL            | There IS an order            |

**Why does this matter?** You can calculate the 'average' of Age (numerical) but NOT the 'average' of Color (categorical). You can rank T-shirt sizes S < M < L (ordinal) but you can't rank cities (nominal). Each type demands its own analysis approach and chart type.

### Quick Decision Rule:

- Can you do math on it (add, average, etc.)? → It's **Numerical**
- Is it a label, name, or group? → It's **Categorical**
- Can you sort/rank the categories? → It's **Ordinal**
- No ranking possible? → It's **Nominal**

## SECTION 04

# Python Packages for EDA

Think of Python packages like **apps on your phone**. Your phone alone can't do much, but with the right apps, it can do anything. Here are the 6 essential packages:

**Pandas**

The Spreadsheet

Load CSVs, filter rows, group data, handle missing values. If Excel was a Python library, this is it. You'll use this in EVERY project.

**Code:** `import pandas as pd`

**NumPy**

The Calculator

Fast math on arrays. Mean, median, standard deviation — all the number crunching. Pandas is built on top of NumPy.

**Code:** `import numpy as np`

**Matplotlib**

The Basic Drawing Tool

Create any chart — line, bar, scatter, pie. Old-school and verbose but gives you full control. The grandfather of Python plotting.

**Code:** `import matplotlib.pyplot as plt`

**Seaborn**

The Pretty Drawing Tool

Built on Matplotlib but makes beautiful charts with less code. Heatmaps, violin plots, pair plots — all in one line. Your go-to for EDA visuals.

**Code:** `import seaborn as sns`

**Plotly**

The Interactive Tool

Charts you can hover, zoom, and click. Creates web-ready interactive visualizations. Great for dashboards and presentations.

**Code:** `import plotly.express as px`

**Scikit-learn**

The AI Brain

Not just for EDA — it's for building ML models too. During EDA, you'll use it for scaling, encoding, and splitting data.

**Code:** `from sklearn import ...`

**Memory trick:** Pandas loads the data, NumPy does the math, Matplotlib/Seaborn draw the pictures, Plotly makes them interactive, Scikit-learn prepares data for AI.

## The starter code for 90% of all data science projects:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

SECTION 05

# List of Graphs

Graphs turn boring numbers into visual stories. But each graph has ONE job. Using the wrong graph is like using a hammer to cut paper. Here's your complete reference:

| Chart Type   | Use When...                          | Data Type               | Example                 | Seaborn Code                         |
|--------------|--------------------------------------|-------------------------|-------------------------|--------------------------------------|
| Bar Chart    | Comparing categories                 | Categorical             | Sales by country        | <code>sns.countplot(x='col')</code>  |
| Line Chart   | Change over time                     | Numerical (time)        | Stock prices monthly    | <code>sns.lineplot(x, y)</code>      |
| Scatter Plot | Relationship between 2 numbers       | Numerical vs Numerical  | Height vs Weight        | <code>sns.scatterplot(x, y)</code>   |
| Histogram    | Distribution / shape of one number   | Numerical               | How are ages spread?    | <code>sns.histplot(df['col'])</code> |
| Pie Chart    | Parts of a whole (%)                 | Categorical             | Market share            | <code>plt.pie(values)</code>         |
| Box Plot     | Spotting outliers, comparing spreads | Numerical               | Salary by department    | <code>sns.boxplot(x, y)</code>       |
| Heatmap      | Correlations between many features   | Numerical grid          | Which features connect? | <code>sns.heatmap(df.corr())</code>  |
| Count Plot   | Counting occurrences                 | Categorical             | Males vs Females count  | <code>sns.countplot(x='col')</code>  |
| Pair Plot    | All relationships at once            | Multiple Numerical      | Full EDA overview       | <code>sns.pairplot(df)</code>        |
| Violin Plot  | Distribution + box plot combined     | Numerical + Categorical | Age dist by class       | <code>sns.violinplot(x, y)</code>    |

## The Golden Rule for choosing graphs:

| Your Question                                | Best Graph            |
|----------------------------------------------|-----------------------|
| Want to COMPARE categories?                  | Bar Chart             |
| Want to see CHANGE OVER TIME?                | Line Chart            |
| Want to find a RELATIONSHIP?                 | Scatter Plot          |
| Want to see the SHAPE / SPREAD?              | Histogram or Box Plot |
| Want to see PROPORTIONS?                     | Pie Chart             |
| Want to find CONNECTIONS between everything? | Heatmap               |

SECTION 06

# Practical EDA — Titanic Walkthrough

Now let's put everything together with a real dataset. The Titanic dataset contains information about 891 passengers — who survived and who didn't. Follow these 7 steps **every time** you do EDA:

## About the Titanic Dataset

| Column      | Type              | Description                     |
|-------------|-------------------|---------------------------------|
| PassengerId | Numerical         | Unique ID for each passenger    |
| Survived    | Categorical (0/1) | 0 = Died, 1 = Survived          |
| Pclass      | Ordinal (1, 2, 3) | Ticket class: 1st, 2nd, 3rd     |
| Name        | Text              | Passenger name                  |
| Sex         | Categorical       | male / female                   |
| Age         | Continuous        | Age in years (has 177 missing!) |
| SibSp       | Discrete          | # of siblings/spouses aboard    |
| Parch       | Discrete          | # of parents/children aboard    |
| Fare        | Continuous        | Ticket price in British Pounds  |
| Cabin       | Categorical       | Cabin number (687 missing!)     |
| Embarked    | Categorical       | Port: C, Q, or S                |



## Step 1: Load the Data

First, load the CSV file into a Pandas DataFrame.

```
import pandas as pd
df = pd.read_csv('titanic.csv')
print(df.shape) # (891, 12) = 891 rows, 12 columns
df.head() # Shows first 5 rows
```

**Insight:** `df.shape` tells you the size instantly. `df.head()` shows a preview. Always start here.

## Step 2: Understand the Structure

Get a bird's-eye view of your data's structure and statistics.

```
df.info() # Column names, types, non-null counts
df.describe() # Mean, min, max, std for numbers
df.dtypes # What type is each column?
```

**Insight:** `df.info()` is your best friend — it reveals column types and how many values are missing. `df.describe()` gives instant statistics. This takes 10 seconds and saves hours.

## Step 3: Find Missing Data

Missing data is like holes in a puzzle. You must find and fix them.

```
# Count missing values per column
df.isnull().sum() # Result: # Age 177 (20% missing) # Cabin 687 (77% missing!) # Embarked 2
# Visualize missing data
import seaborn as sns
sns.heatmap(df.isnull(), cbar=True, yticklabels=False)
```

**Insight:** Age has 177 missing — fill with median. Cabin has 687 missing (77%) — probably drop it. The heatmap shows missing values as bright lines.

## Step 4: Analyze Numerical Features

Use histograms, box plots, and correlation heatmaps for numbers.

```
# Distribution of Age
df['Age'].hist(bins=30) # Outlier detection with box plot
df['Fare'].plot(kind='box') # Correlation between all numbers
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

**Insight:** Histograms reveal most passengers are aged 20-35. Box plots show extreme fares (rich passengers!). The correlation heatmap shows Fare and Pclass are strongly linked.

## Step 5: Analyze Categorical Features

Use count plots and groupby for categories — this is where stories hide!

```
# Count each category df['Sex'].value_counts() # male: 577, female: 314 # Survival rate by gender df.groupby('Sex')['Survived'].mean() # female: 0.74 (74% survived!) # male: 0.19 (only 19%) # Visual comparison sns.countplot(x='Pclass', hue='Survived', data=df)
```

**Insight:** The shocking discovery: 74% of women survived but only 19% of men — the 'women and children first' policy in action. 1st class survived 2.5x more than 3rd class.

## Step 6: Feature Engineering

Create NEW columns from existing data to find deeper patterns.

```
# Create FamilySize df['FamilySize'] = df['SibSp'] + df['Parch'] + 1 # Create IsAlone flag df['IsAlone'] = (df['FamilySize'] == 1).astype(int) # Extract Title from Name df['Title'] = df['Name'].str.extract(r' ([A-Za-z]+)\.') # Analyze: do lone travelers survive less? df.groupby('IsAlone')['Survived'].mean() # Alone: 0.30 | With family: 0.50
```

**Insight:** People with families survived 50% vs 30% for loners. Titles like 'Mrs' and 'Miss' had much higher survival than 'Mr'. New features often predict better than originals!

## Step 7: Summarize Your Insights

Write down everything you discovered. These guide your ML model.

```
# Your EDA conclusions: # 1. Women survived 3.7x more than men # 2. 1st class survived 2.5x more than 3rd # 3. Children (age < 12) had higher survival # 4. Solo travelers were more vulnerable # 5. Higher fare = higher survival chance # 6. Age has 20% missing -> fill with median # 7. Cabin has 77% missing -> drop it # 8. New features: FamilySize, IsAlone, Title
```

**Insight:** You now know which features matter (Sex, Pclass, Age, Fare), what to fix (missing Age), what to drop (Cabin), and what to create (FamilySize, IsAlone). You are ready to build a model!

SECTION BONUS

# Google Colab — Your Free AI Lab

**Google Colab** (short for Google Colaboratory) is like **Google Docs**, but for writing **Python code**. Just like you can write a document in Google Docs without installing anything, you can write and run Python code in Google Colab without installing Python on your computer.

| Feature       | Details                                                               |
|---------------|-----------------------------------------------------------------------|
| Cost          | Completely FREE to start                                              |
| Installation  | NONE needed — runs in your browser                                    |
| Pre-installed | Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn — all ready to go    |
| GPU Access    | Free GPU for deep learning (Runtime → Change runtime type → GPU)      |
| Save Location | Auto-saves to your Google Drive                                       |
| Sharing       | Share with a link, just like Google Docs                              |
| Time Limit    | Free tier: ~90 min idle timeout, 12-hour session max                  |
| Who uses it   | Students, researchers, Kaggle competitors, AI professionals worldwide |

## How to Get Started (30 seconds):

| Step | Action                                                                          |
|------|---------------------------------------------------------------------------------|
| 1    | Open your browser (Chrome recommended)                                          |
| 2    | Go to <a href="https://colab.research.google.com">colab.research.google.com</a> |
| 3    | Sign in with your Gmail account                                                 |
| 4    | Click 'New Notebook'                                                            |
| 5    | Type: <code>print("Hello, AI world!")</code> in the first cell                  |
| 6    | Press Shift + Enter                                                             |
| 7    | You're now coding in Python!                                                    |

## The Analogy:

**Your laptop** = a small kitchen. You can make a sandwich, but cooking a feast for 100 people? Your stove isn't big enough. **Google Colab** = a giant professional kitchen that Google lets you use for free. You walk in, cook whatever you want, and walk out. Google handles the expensive equipment.

**Pro tip:** For your 21-day AI journey, the free Colab tier is more than enough. If you ever need more power, Colab Pro costs about \$10/month — but you won't need it yet.

# EDA Cheat Sheet

Tear out this page and keep it next to your screen. This is everything you need to remember:

## The 7-Step EDA Recipe

| # | Step                 | Key Command                                                  | What You Get                |
|---|----------------------|--------------------------------------------------------------|-----------------------------|
| 1 | Load data            | <code>df = pd.read_csv('file.csv')</code>                    | DataFrame in memory         |
| 2 | Understand structure | <code>df.info()</code> , <code>df.describe()</code>          | Types, stats, shape         |
| 3 | Find missing data    | <code>df.isnull().sum()</code>                               | Missing value counts        |
| 4 | Analyze numbers      | <code>df.hist()</code> , <code>sns.heatmap(df.corr())</code> | Distributions, correlations |
| 5 | Analyze categories   | <code>df['col'].value_counts()</code>                        | Counts, survival rates      |
| 6 | Engineer features    | <code>df['new'] = df['a'] + df['b']</code>                   | Better predictors           |
| 7 | Summarize insights   | Write bullet points                                          | Action plan for modeling    |

## Data Type Decision Tree

| Ask Yourself           | Answer                          | Data Type                        |
|------------------------|---------------------------------|----------------------------------|
| Can you do math on it? | YES → Is it whole numbers only? |                                  |
|                        | YES → Discrete                  | Discrete (e.g., 0, 1, 2, 3)      |
|                        | NO → Continuous                 | Continuous (e.g., 5.7, 98.6)     |
| Can you do math on it? | NO → Is there a ranking/order?  |                                  |
|                        | YES → Ordinal                   | Ordinal (e.g., Low, Med, High)   |
|                        | NO → Nominal                    | Nominal (e.g., Red, Blue, Green) |

## Graph Selection Guide

| Your Goal                    | Graph        | One-liner Code                                        |
|------------------------------|--------------|-------------------------------------------------------|
| Compare categories           | Bar Chart    | <code>sns.countplot(x='col', data=df)</code>          |
| Change over time             | Line Chart   | <code>sns.lineplot(x='date', y='val', data=df)</code> |
| Relationship of 2 numbers    | Scatter Plot | <code>sns.scatterplot(x='a', y='b', data=df)</code>   |
| Shape / spread of 1 number   | Histogram    | <code>sns.histplot(df['col'], bins=30)</code>         |
| Parts of whole (%)           | Pie Chart    | <code>df['col'].value_counts().plot.pie()</code>      |
| Outliers + spread comparison | Box Plot     | <code>sns.boxplot(x='cat', y='num', data=df)</code>   |
| All correlations at once     | Heatmap      | <code>sns.heatmap(df.corr(), annot=True)</code>       |
| Everything vs everything     | Pair Plot    | <code>sns.pairplot(df)</code>                         |

## Essential Imports

```
import pandas as pd # Load and manipulate data
import numpy as np # Math operations
import matplotlib.pyplot as plt # Basic charts
import seaborn as sns # Beautiful charts
```

**You now have everything you need to master EDA.**  
**Open Google Colab, load the Titanic dataset, and start exploring!**

**21 Days. 21 Projects. Infinite Possibilities.**